

Arreglos

Rotar un arreglo

Diseñe un algoritmo que reciba como entrada un arreglo de enteros A de largo N y un entero positivo K. El algoritmo debe rotar el arreglo K posiciones a la derecha de manera circular, dicho algoritmo no debe crear un arreglo auxiliar nuevo, sino que realizar todo in place.

```
Ejemplo 1:
  Input:
    A = {1, 2, 3, 6, 7}
    K = 2
  Output:
    {6, 7, 1, 2, 3}

Ejemplo 2:
  Input:
    A = {1, 2, 3, 4, 5, 6}
    K = 7
  Output:
    {6, 1, 2, 3, 4, 5}
```

Rotar una matriz

Diseñe un algoritmo que reciba como entrada una matriz de enteros M con dimensiones $N \times N$. Su algoritmo debe rotar la matriz M en 90 grados en sentido del reloj. La operación de rotación debe ser in place, es decir no puede crear una matriz auxiliar.

```
Ejemplo 1:
  Input:
    M = {{1,2,3},
          {4,5,6},
          {7,8,9}}
  Output:
    {{7,4,1},
     {8,5,2},
     {9,6,3}}

Ejemplo 2:
  Input:
    M = {{1, 2, 3, 4},
          {5, 6, 7, 8},
          {9, 10, 11, 12},
          {13, 14, 15, 16}}
  Output:
    {{13, 9, 5, 1},
     {14, 10, 6, 2},
     {15, 11, 7, 3},
     {16, 12, 8, 4}}
```

Invertir un arreglo

Considere el problema de invertir un arreglo de enteros. Diseñe e implemente las siguientes variaciones el algoritmo:

1. Diseñe un algoritmo que utilice un stack como estructura de datos auxiliar para invertir el arreglo.
2. Diseñe un algoritmo que invierta el arreglo de forma recursiva, sin utilizar un stack como estructura de datos auxiliar.
3. Diseñe un algoritmo que invierta el arreglo de forma iterativa, sin usar un stack como estructura de datos auxiliar.

Ejemplo:

Input:

A = {1, 2, 3, 6, 7}

Output:

A' = {7, 6, 3, 2, 1}

Listas Enlazadas

Mezclar dos listas enlazadas ordenadas

Sean dos listas enlazadas, ambas definidas como:

```
1 class LinkedList {  
2     class Node{  
3         int value;  
4         Node next;  
5     }  
6     Node Head;  
7 }
```

Diseñe el método mezclar listas enlazadas, que recibe como entrada el head de dos listas enlazadas ordenadas de manera ascendente y retorna como salida el head de una nueva lista enlazada que mezcla ambas listas de manera ordenada.

Ejemplo:

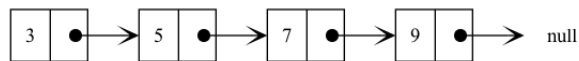


Figure 1: Entrada: Lista enlazada 1



Figure 2: Entrada: Lista enlazada 2



Figure 3: Output: Lista enlazada mezclada

Intercambiar nodos en pares

Considere el problema de intercambiar de posición todos los pares de nodos adyacentes en una lista enlazada. El algoritmo como entrada recibe el head de una lista enlazada L y debe intercambiar los nodos de posición in place.

```
Ejemplo 1:  
  Input:  
    L = 1 -> 2 -> 3 -> 4  
  Output:  
    2 -> 1 -> 4 -> 3  
  
Ejemplo 2:  
  Input:  
    L = 1 -> 2 -> 3 -> 4 -> 5  
  Output:  
    2 -> 1 -> 4 -> 3 -> 5
```

Invertir una lista enlazada

Sea una lista enlazada definida:

```
1 class LinkedList {  
2     class Node{  
3         int value;  
4         Node next;  
5     }  
6     Node Head;  
7 }
```

Diseñe el método invertir lista, que recibe el Head de una lista y la deja en orden inverso.

1. Diseñe el método invertir de forma iterativa utilizando un stack como estructura de datos auxiliar.
2. Diseñe el método invertir de forma iterativa sin utilizar un stack como estructura de datos auxiliar.
3. Diseñe el método invertir de forma recursiva.

```
Ejemplo:  
  Input:  
    L = 1 -> 2 -> 3 -> 4  
  Output:  
    4 -> 3 -> 2 -> 1
```

Stacks

Paréntesis balanceado

Diseñe un algoritmo que recibe como entrada un string que puede estar compuesto por los siguientes caracteres:

```
[ ] ( ) { }
```

Como output el algoritmo debe retornar un booleano que describa si la expresión formada es válida, es decir que para cada símbolo de apertura, existe un símbolo de cierre en el lugar correcto.

```
Ejemplo 1:  
  Input:  
    { ( ) [ ] }  
  Output:  
    true  
Ejemplo 2:  
  Input:  
    { } ( ) [ ( ]  
  Output:  
    false  
Ejemplo 3:  
  Input:  
    { } ( [ { } [ ] ( { [ ] } ) ] )  
  Output:  
    true
```

Notación de postfijo

Diseñe un algoritmo que recibe una expresión aritmética de postfijo y calcula el resultado. En una expresión de postfijo los operadores se escriben después del operando, como por ejemplo:

2 3 + 3 3 + *

Dicho ejemplo se puede interpretar(en notación de infijo) como:

(2 + 3)(3 + 3)

Diseñe un algoritmo que interpreta una expresión de postfijo y retorna el valor correspondiente.

```
Ejemplo 1:  
  Input:  
    2 3 + 3 3 + *  
  Output:  
    30  
Ejemplo 2:  
  Input:  
    9 9 *  
  Output:  
    81  
Ejemplo 3:  
  Input:  
    7 7 * 9 3 / +  
  Output:  
    52
```

Cola

Cajas en un banco

Considere una sala de atención al cliente en un banco que tiene solo dos cajas de atención, una caja para creación y eliminación de productos y otra caja para reportar problemas en algún producto. Considere que el tiempo de atención en cada caja es el mismo tiempo t .

Diseñe un algoritmo que recibe un string compuesto por los caracteres C, E, P, donde cada caracter denota:

C: Crear producto
P: Reportar problema
E: Eliminar producto

Como output su algoritmo debe retornar que caja termina de atender primero a una fila de clientes. La caja de creación y eliminación de clientes es denotada por c_1 y la caja de reporte de problemas por c_2 , en caso que ambas cajas terminen al mismo tiempo retorne c_1c_2 .

```
Ejemplo 1:  
  Input:  
    C C E P P  
  Output:  
    C2  
Ejemplo 2:  
  Input:  
    C P E E C P P C P P C P P P  
  Output:  
    C1  
Ejemplo 3:  
  Input:  
    C P E P  
  Output:  
    C1C2
```

Cola de impresión

En Toogle se quedaron sin plata para comprar pantallas, así que decidieron que van a volver a programar a la antigua; imprimiendo el código en una impresora. Para esto empezarán con una cola de impresión experimental, en la cual cada programador puede mandar a imprimir el código en el que está trabajando, ya sea para fines de refactorizar, debuggear o rediseñar algún componente del sistema. Aunque para esta cola de impresión decidieron tomar en cuenta la aparición de incidentes de carácter urgente que su equipo de SRE necesite resolver, es por eso que la cola de impresión permitirá agregar al inicio de la cola los trabajos de impresión que sean urgentes.

Implemente el TDA `PrintQueue` que funciona de la siguiente forma:

1. Permite imprimir en orden FIFO, es decir los trabajos de impresión se van ejecutando para los primeros trabajos que entraron.
2. Permite imprimir trabajos de manera urgente, es decir que si un trabajo de impresión tiene carácter de urgente se imprimirá apenas la impresora esté disponible para imprimir un trabajo nuevo.
3. La impresora demora un tiempo constante t multiplicado por la cantidad de páginas p_i en el trabajo i , es decir por cada trabajo demora $p_i \times t$.

Diseñe e implemente la interfaz `PrintQueue` para que cumpla con la siguiente interface, no hay reglas sobre cómo implemente la interfaz respetando la definición de los siguientes métodos:

1. `Encolar`: método que recibe como entrada un trabajo de impresión, este trabajo de impresión almacena la cantidad de páginas a imprimir.
2. `EncolarUrgente`: método que recibe como entrada un trabajo de impresión urgente, este trabajo de impresión almacena la cantidad de páginas a imprimir. Este método debe garantizar que el trabajo urgente será el próximo a imprimir después del trabajo actual. Los trabajos urgentes tienen la prioridad por sobre los trabajos normales. Si hay más de un trabajo urgente a la espera, entonces estos se imprimen en orden FIFO.
3. `Imprimir`: imprime la cantidad de páginas del trabajo i .
4. `Historial de impresiones`: retorna una lista histórica de las impresiones enumeradas en orden LIFO.

A modo de ejercicio, diseñe un conjunto de casos de pruebas para poner a prueba la correctitud y eficiencia de la `PrintQueue` que diseñen e implementen.

Compilador a papel

Para cada uno de los siguientes algoritmos, responda las preguntas propuestas

Torres de Hanoi

```
1 public static void hanoi(int n, char src, char to, char aux) {  
2     if (n == 1) {  
3         System.out.println("Move disk 1 src rod " + src + " to rod " + to);  
4         return;  
5     }  
6     hanoi(n - 1, src, aux, to);  
7     System.out.println(  
8         "Move disk " + n + " src rod " + src + " to rod " + to  
9     );  
10    hanoi(n - 1, aux, to, src);  
11 }
```

Listing 1: Tower of Hanoi

1. Ejecute a mano el algoritmo hanoi con input:

```
n = 3  
src = 'A'  
to = 'C'  
aux = 'B'
```

2. Analice el tiempo de ejecución del algoritmo utilizando la notación Big O.

Gray Codes

```
1 public static List<String> generateGrayCode(int n) {  
2     if (n == 0) {  
3         List<String> baseCase = new ArrayList<>();  
4         baseCase.add("");  
5         return baseCase;  
6     }  
7  
8     List<String> prevGrayCode = generateGrayCode(n - 1);  
9     List<String> grayCode = new ArrayList<>();  
10  
11     for (String code : prevGrayCode) {  
12         grayCode.add("0" + code);  
13     }  
14  
15     for (int i = prevGrayCode.size() - 1; i >= 0; i--) {  
16         grayCode.add("1" + prevGrayCode.get(i));  
17     }  
18  
19     return grayCode;  
20 }
```

Listing 2: Gray Codes

1. Ejecute el algoritmo generateGrayCode con input $n = 4$.
2. Analice el tiempo de ejecución del algoritmo utilizando la notación Big O.

Greatest Common Divisor

```
1 public static int gcd(int N, int M) {  
2     if (M == 0) {  
3         return N;  
4     }  
5     return gcd(M, N % M);  
6 }
```

Listing 3: Greatest Common Divisor

1. Ejecute a mano el algoritmo GCD con input

N = 2
M = 512

2. Ejecute a mano el algoritmo GCD con input

N = 81
M = 3

3. Ejecute a mano el algoritmo GCD con input

N = 211
M = 37

4. Analice el tiempo de ejecución del algoritmo utilizando la notación Big O.

Análisis

Para cada uno de los siguientes algoritmos, analice el tiempo de ejecución utilizando la notación Big O.

```
1. public static int f1(int N) {  
2     if (N == 0) return 0;  
3     return f1(N - 1) + 1;  
4 }  
5
```

Listing 4: f1

```
2. public static int f2(int N) {  
2     if (N == 0) return 0;  
3     return N + f2(N - 1);  
4 }  
5
```

Listing 5: f2

```
3. public static int f3(int N) {  
2     if (N == 0) return 1;  
3     return N * f3(N - 1);  
4 }  
5
```

Listing 6: f3

```
4. public static int f4(int N) {  
2     if (N == 0) return 0;  
3     int sum = 0;  
4     for (int i = 1; i <= N; i++) {  
5         sum += f4(N - i) + 1;  
6     }  
7     return sum;  
8 }  
9
```

Listing 7: f4

```
5. public static void f5(int N) {  
2     if (N == 0) return;  
3     f5(N - 1);  
4     f5(N - 1);  
5 }  
6
```

Listing 8: f5

```
6. public static void f6(int i, int N) {  
2     if (i * i >= N || i <= 0) return;  
3     f6(i + 1, N);  
4 }  
5
```

Listing 9: f6

```
7. public static void f7(int N) {  
2     if (N == 0) return;  
3     for (int i = 0; i < N; i++) {  
4         f7(N - 1);  
5     }  
6 }  
7
```

Listing 10: f7

```
8. public static int f8(int N) {  
2     if (N == 0) return 0;  
3     return f8(N / 2) + 1;  
4 }  
5
```

Listing 11: f8

```
9. public static int f9(int N) {  
2     if (N == 0) return 0;  
3     int sum = 0;  
4     for (int i = 1; i <= N; i++) {  
5         for (int j = 1; j <= N; j++) {  
6             sum += f9(N - i);  
7         }  
8     }  
9     return sum;  
10 }  
11
```

Listing 12: f9